

Templates : `Template/PG-fr.docx` | `Template/Presentation_BELACEL_V3.pptx`

PDF : `TP_Informatique_ENS_1ère_année_AN_06.pdf`

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS — Year 1 — English Department

Duration: 1h30

Academic Year: 2025-2026

Learning Objectives

At the end of this lab, the student is able to:

- Implement a stack (LIFO) in Python using a list
- Implement a queue (FIFO) in Python using `collections.deque`
- Use push/pop and enqueue/dequeue operations
- Solve classic problems with stacks and queues
- Understand the usefulness of data structures in algorithmics
- Implement a calculator in Reverse Polish Notation (RPN)

Lab Plan

Phase	Duration	Activity
Welcome and review	5 min	LIFO/FIFO analogies
Exercise 1	15 min	Stack implementation
Exercise 2	10 min	Queue implementation
Exercise 3	20 min	Bracket validation
Exercise 4	20 min	Print queue
Exercise 5	20 min	Reverse Polish Notation

Exercise 1 — Stack Implementation (LIFO) (15 min)

Problem statement: Create a script `pile.py` that implements a stack using a Python list. Define the functions:

1. `push(stack, element)` — adds to the top
2. `pop(stack)` — removes and returns the top
3. `peek(stack)` — returns the top without removing
4. `size(stack)` — returns the number of elements
5. `display(stack)` — shows the content

Test with: push 10, 20, 30, pop once, push 40, display.

```

flowchart LR
    subgraph Stack[LIFO Stack]
        direction TB
        S1["30 ← TOP"] --> S2["20"]
        S2 --> S3["10"]
        S3 --> S4["BOTTOM"]
    end
    E[push 40] --> P2
    subgraph P2[Stack after push 40]
        direction TB
        T1["40 ← TOP"] --> T2["20"]
        T2 --> T3["10"]
        T3 --> T4["BOTTOM"]
    end
end

```

Solution:

```

def push(stack, element):
    stack.append(element)

def pop(stack):
    if stack:
        return stack.pop()
    return None

def peek(stack):
    if stack:
        return stack[-1]
    return None

def size(stack):
    return len(stack)

def display(stack):
    print(f"Stack: {stack} (top → {peek(stack)})")

p = []
push(p, 10)
push(p, 20)
push(p, 30)
print(f"Before pop: {p}")
x = pop(p)
print(f"Popped: {x}")
push(p, 40)
display(p)
print(f"Size: {size(p)}")

```

Exercise 2 — Queue Implementation (FIFO) (10 min)

Problem statement: Create a script file.py that implements a queue using collections.deque. Define the functions:

1. enqueue(queue, element) — adds to the end
2. dequeue(queue) — removes and returns the front
3. front(queue) — returns the front without removing
4. display(queue) — shows the content

```

flowchart LR
    subgraph Queue[FIFO Queue]
        direction LR
        D1["FRONT → A"] --> D2["B"] --> D3["C"] --> D4["REAR"]
    end

```

```

end
action["dequeue → A leaves"] --> F2
subgraph F2[Queue after dequeue]
direction LR
E1["FRONT → B"] --> E2["C"] --> E3["REAR"]
end

```

Solution:

```

from collections import deque

def enqueue(queue, element):
    queue.append(element)

def dequeue(queue):
    if queue:
        return queue.popleft()
    return None

def front(queue):
    if queue:
        return queue[0]
    return None

def display(queue):
    print(f"Queue: {list(queue)} (front → {front(queue)})")

f = deque()
enqueue(f, "A")
enqueue(f, "B")
enqueue(f, "C")
print(f"Before dequeue: {list(f)}")
x = dequeue(f)
print(f"Dequeued: {x}")
enqueue(f, "D")
display(f)

```

Exercise 3 — Balanced Parentheses Checker (20 min)

Problem statement: Create a script `parentheses.py` that checks whether a mathematical expression is correctly parenthesized. The program must handle `()`, `[]`, `{}`.

Examples:

- $(3 + 2) \times [5 - \{1 + 2\}] \rightarrow \text{Correct}$
- $(3 + 2) \times [5 - \{1 + 2\}) \rightarrow \text{Incorrect}$
- $(\{[]\}) \rightarrow \text{Incorrect}$

```

flowchart TD
    A[Expression] --> B[Initialize empty stack]
    B --> C[Loop through each character]
    C --> D{Opening character?}
    D -->|Yes| E[Push]
    E --> C
    D -->|No| F{Closing character?}
    F -->|No| C
    F -->|Yes| G{Stack empty?}
    G -->|Yes| H[Incorrect]
    G -->|No| I[Pop and compare]
    I --> J{Match?}
    J -->|No| H

```

```

J -->|Yes| C
C --> K[End of traversal]
K --> L{Stack empty?}
L -->|Yes| M[Correct ✓]
L -->|No| H

```

Solution:

```

def is_correct(expression):
    """Checks the parenthesization of an expression."""
    matching = {"(": ")", "[": "]", "{": "}"
    stack = []

    for char in expression:
        if char in "([{":
            stack.append(char)
        elif char in ")]}":
            if not stack or stack.pop() != matching[char]:
                return False

    return len(stack) == 0

expressions = [
    "(3 + 2) × [5 - {1 + 2}]",
    "(3 + 2] × [5 - {1 + 2})",
    "((1 + 2) × 3)",
    "({[]})"
]

for expr in expressions:
    state = "✓ Correct" if is_correct(expr) else "× Incorrect"
    print(f"{state} {expr}")

```

Exercise 4 — Print Queue Management (20 min)

Problem statement: Create a script file `impression.py` that simulates a print queue. Each document has a name and a number of pages. The program:

1. Adds documents to the queue
2. "Prints" each document (displays name + pages)
3. Displays the total waiting time and per document (1 page = 2 seconds)

Expected example:

```

Added: Rapport_TP5.pdf (15 pages)
Added: Cours_Architecture.pdf (42 pages)
Added: Devoir_Maison.docx (8 pages)

Document 1: Rapport_TP5.pdf – 15 pages (print time: 30s, cumulative wait: 30s)
Document 2: Cours_Architecture.pdf – 42 pages (print time: 84s, cumulative wait: 114s)
Document 3: Devoir_Maison.docx – 8 pages (print time: 16s, cumulative wait: 130s)

Total print time: 130s

```

Solution:

```

from collections import deque

print_queue = deque()

def add_document(name, pages):

```

```

"""Adds a document to the print queue."""
doc = {"name": name, "pages": pages}
print_queue.append(doc)
print(f"Added: {name} ({pages} pages)")

def print_all():
    """Simulates printing all documents."""
    total_time = 0
    number = 1
    while print_queue:
        doc = print_queue.popleft()
        time = doc["pages"] * 2
        total_time += time
        print(f"Document {number}: {doc['name']} - {doc['pages']} pages "
              f"(print time: {time}s, cumulative wait: {total_time}s)")
        number += 1
    print(f"\nTotal print time: {total_time}s")

add_document("Rapport_TP5.pdf", 15)
add_document("Cours_Architecture.pdf", 42)
add_document("Devoir_Maison.docx", 8)
print()
print_all()

```

Exercise 5 — Reverse Polish Notation (Calculator) (20 min)

Problem statement: Create a script `npi.py` that implements a calculator in Reverse Polish Notation (RPN). In RPN, the operator comes after the operands: `3 4 +` means `3 + 4`. Use a stack to evaluate the expression.

Features:

1. Read an RPN expression (operands and operators separated by spaces)
2. Use a stack: push numbers, pop for operations
3. Handle `+`, `-`, `*`, `/`
4. Handle errors (division by zero, invalid expression)

Expected example:

RPN expression: `3 4 + 2 *`
 Result: 14.0

Explanation: `3 4 +` → 7, then `7 2 *` → 14

Solution:

```

def evaluate_rpn(expression):
    """Evaluates an expression in Reverse Polish Notation."""
    stack = []
    elements = expression.strip().split()

    for elem in elements:
        if elem in "+-*/":
            if len(stack) < 2:
                return "Error: invalid expression"
            b = stack.pop()
            a = stack.pop()
            if elem == "+":
                stack.append(a + b)
            elif elem == "-":

```

```

stack.append(a - b)
elif elem == "*":
    stack.append(a * b)
elif elem == "/":
    if b == 0:
        return "Error: division by zero"
    stack.append(a / b)
else:
    try:
        stack.append(float(elem))
    except ValueError:
        return f"Error: unknown element '{elem}'"

if len(stack) == 1:
    return stack[0]
return "Error: invalid expression"

# Tests
expressions = [
    "3 4 + 2 *",
    "10 5 /",
    "5 1 2 + 4 * + 3 -",
    "4 0 /"
]

for expr in expressions:
    result = evaluate_rpn(expr)
    print(f"{expr} → {result}")

```

Self-Assessment MCQ

- What does LIFO mean?
 - Last In, First Out ✓
 - Large In, Fast Out
 - Last Input, Fast Operation
- Which data structure should be used for a cafeteria queue?
 - A stack
 - A queue ✓
 - A dictionary
- In Python, how to simply implement a stack?
 - With a dictionary
 - With a list and the `append()`/`pop()` methods ✓
 - With a tuple
- What does `pop()` return on an empty list?
 - An error (`IndexError`) ✓
 - None
 - 0
- In RPN, what does `5 2 - 3 *` equal?

- a) -1
- b) 9 ✓
- c) 21

Grading Rubric — Lab 06

Criterion	Weight	Not acquired (0)	Partial (0.5)	Acquired (1)
Stack implementation (LIFO)	20%	Cannot implement	Partially correct operations	Correct implementation
Queue implementation (FIFO)	20%	Cannot implement	Partially correct operations	Correct implementation
Parenthesization algorithm	20%	No algorithm	Partial algorithm	Correct algorithm
Print queue simulation	20%	No simulation	Partial simulation	Complete simulation
RPN calculator	20%	No calculator	Partially implemented RPN	Functional RPN

Session Schedule — 1h30 (90 min)

Phase	Duration	Cumulative total
Welcome	5 min	5 min
Exercise 1 — Stack	15 min	20 min
Exercise 2 — Queue	10 min	30 min
Exercise 3 — Parenthesization	20 min	50 min
Exercise 4 — Print queue	20 min	70 min
Exercise 5 — RPN calculator	20 min	90 min
Total	90 min	

Pedagogical Note ENS

Teaching transposition

Stacks and queues are fundamental data structures in computer science. This lab addresses them progressively:

- Concrete analogies: Before coding, use physical objects. A stack of plates (LIFO) — you put and take from the top. A cafeteria queue (FIFO) — first come, first served.
- Exercise 3 (parenthesization): This is a classic from the NSI baccalaureate exam. Show how a simple stack can solve a complex syntactic validation problem.

Trainer advice

Have students physically act out the simulations in class before coding. One group of students forms a queue, another a stack. The operations are mimed. This unplugged approach is recommended for all levels from middle to high school. For NSI students, add handling of the three bracket types `()`, `[]`, `{}`.

Extension

Implement a Reverse Polish Notation (RPN) calculator that uses a stack to evaluate expressions like $3\ 4\ +\ 2\ *$ (result: 14). This extension prepares for session 10 of the module.